Institut Supérieur d'Informatique et de Mathématiques de Monastir (ISIMM)

Academic year 2026/2027

Sentiment Analysis of Movie Reviews

Phase 1: business understanding, data collection,
data understanding and data preparation

Prepared by: Mohamed Yassine Saidane

November 8, 2026

Abstract

This report documents the first phase of a sentiment analysis project on English movie reviews: business understanding, data collection, data understanding and data preparation. The goal is a system that classifies a review as positive or negative, so that studios, platforms and critics can measure audience reception without reading thousands of reviews. Three datasets are used: the IMDB 50k benchmark (49,582 reviews after removing 418 duplicates), the Pang & Lee polarity corpus (2,000 reviews) as an external test set, and 300 recent reviews collected through the TMDB API and labeled from the reviewers' ratings. A second annotator confirmed these labels on a 50-review sample (Cohen's $\kappa = 0.96$). The exploration shows balanced classes, an average length of 231 words in IMDB (127 in our own reviews, 746 in Pang & Lee), HTML noise in 58% of the reviews and a negation word in 87% of them. A cleaning pipeline that preserves negations and lemmatizes the text was selected after comparing five variants with a TF-IDF and logistic regression baseline, which reached a validation macro-F1 of 0.916 with unigrams and bigrams. The phase produces clean, leakage-free training, validation and test files, ready for feature engineering and modeling.

Contents

1 Introduction **2**
- 1.1 Context . . . 2
- 1.2 Problem statement . . . 2
- 1.3 Objectives of the project . . . 2
- 1.4 Scope of this report . . . 2

2 Business understanding **3**
- 2.1 Problem and business objectives . . . 3
- 2.2 Stakeholders . . . 3
- 2.3 Why data mining and machine learning . . . 3
- 2.4 Data-mining goal . . . 4
- 2.5 Success criteria . . . 4
- 2.6 Scope, assumptions and constraints . . . 4
- 2.7 Risks and mitigations . . . 5
- 2.8 Project plan and resources . . . 5

3 Data collection **6**
- 3.1 Overview of the sources . . . 6
- 3.2 Benchmark: IMDB 50k . . . 6
- 3.3 External set: Pang & Lee . . . 6
- 3.4 Own dataset . . . 7
- 3.5 Ethics and licensing . . . 8
- 3.6 Summary of the datasets . . . 8

4 Data understanding **9**
- 4.1 Structure and quality . . . 9
- 4.2 Review length . . . 9
- 4.3 Noise . . . 10
- 4.4 Vocabulary . . . 10
- 4.5 Negation . . . 10
- 4.6 Comparison of the three datasets . . . 11
- 4.7 Hard cases in the own reviews . . . 11
- 4.8 Summary of findings . . . 11

5 Data preparation **12**
- 5.1 Pipeline overview . . . 12
- 5.2 Deduplication . . . 12
- 5.3 Text cleaning . . . 12
- 5.4 Training, validation and test split . . . 13
- 5.5 Integration of the three sources . . . 13
- 5.6 Enrichment . . . 14
- 5.7 Comparison of cleaning variants . . . 14

6 Conclusion and next steps **15**
- 6.1 Summary of the phase . . . 15
- 6.2 Limitations . . . 15
- 6.3 Next steps . . . 15

A Code and reproduction **17**
- A.1 Preparation code . . . 17
- A.2 Structure of the deliverable . . . 17
- A.3 How to reproduce . . . 18

Chapter 1

Introduction

1.1 Context

Online platforms collect large volumes of free-text reviews. A popular film can receive thousands of them, written with different lengths, styles and levels of detail. Star ratings summarize part of this opinion, but many reviews come without a rating, and a rating hides the reasons behind it. Reading every review is impossible, and a sample is biased.

1.2 Problem statement

Studios, streaming platforms and critics need to know how an audience received a film and how this reception evolves over time. This project builds a system that classifies the sentiment of an English movie review as positive or negative. It follows the data-science cycle CRISP-DM.

1.3 Objectives of the project

- Measure audience reception from review text within seconds.
- Build a clean, reproducible dataset from a benchmark and from independently collected reviews.
- Compare classical and deep models on this data and measure how they behave on independent sets.
- Deliver a working demonstration application.

1.4 Scope of this report

This report covers the first phase of the project, up to data preparation. Chapter 2 presents the business understanding, Chapter 3 the data collection, Chapter 4 the data understanding and Chapter 5 the data preparation. Chapter 6 summarizes the results and presents the next steps. Feature engineering, modeling, evaluation and deployment belong to the next phases. The deliverables are the raw data files, the project code (three notebooks and one module), the folder of plots and this report.

Chapter 2

Business understanding

2.1 Problem and business objectives

The business problem is the cost of reading reviews at scale. The objectives are to:

- estimate the audience reception of a film from the text of its reviews within seconds;
- provide a consistent score that can be compared across films, genres and periods;
- support decisions such as marketing adjustments, recommendation and ranking, and early detection of films that polarize audiences.

2.2 Stakeholders

Stakeholder	Use of the system
Studios and distributors	Track reception after a release and adapt marketing
Streaming platforms	Feed recommendation and ranking, flag poorly received films
Critics and journalists	Summarize public opinion to support an article
Researchers	Reuse the pipeline as a baseline for other domains

Table 2.1: Stakeholders and expected value

2.3 Why data mining and machine learning

Three characteristics of the problem make it a good fit for data mining and machine learning.

- **Unstructured text at scale.** Reviews are free text, and their number grows with the popularity of a film. Manual analysis does not scale.
- **Rules are not enough.** A keyword list (“good”, “bad”) fails on negation (“not good”), sarcasm and mixed opinions (“great cast, awful script”). Handcrafting rules for all such cases is not realistic.
- **Labeled examples exist.** Large collections of reviews come with the reviewer’s own rating, which provides labels for free. A supervised model can learn from these examples and then process new reviews automatically.

2.4 Data-mining goal

The task is supervised binary text classification. The input is the raw text of one English review. The output is a label (positive = 1, negative = 0) and a confidence score. Classical models (TF-IDF with Naive Bayes, Logistic Regression and SVM) will be compared with one deep model in the next phases.

2.5 Success criteria

- **Technical:** macro-F1 ≥ 0.88 on a held-out test set that is never used for tuning; accuracy, precision, recall, F1 and the confusion matrix reported for every model.
- **Generalization:** performance measured on two independent sets (Pang & Lee and the own reviews). A drop is expected, and measuring it is part of the study.
- **Operational:** prediction time under one second per review and a working demonstration application.

Macro-F1 averages the F1 of both classes, so it remains reliable if the classes become unbalanced. On the balanced benchmark it is close to accuracy.

2.6 Scope, assumptions and constraints

In scope: English reviews of films that are clearly positive or clearly negative.

Out of scope: neutral or mixed ratings, other languages, aspect-level sentiment (acting versus plot) and other domains such as hotels.

Assumptions: the rating given by the reviewer reflects the sentiment of the text, so labels are reliable (checked on a sample in Section 3.4); the benchmark is representative enough of film reviews.

Constraints: a one-month timeline, limited hardware (heavy models are trained on a free cloud GPU if needed), and public data only, used for academic purposes with sources cited.

2.7 Risks and mitigations

Risk	Example	Mitigation
Negation	"not good" read as positive	Keep negation words, use bigrams, compare cleaning variants
Sarcasm	"Great, two hours I will never get back"	Flag hard cases in the own dataset and report their error rate
Mixed opinions	"Great cast, awful script"	Bigrams, context-aware models, error analysis
Domain shift	A model trained on movies applied to hotels	External test sets, state the limit clearly
Data leakage	Duplicates, same film in train and test	Remove duplicates before splitting, keep the test set untouched
Label noise	A rating that contradicts the text	Use extreme ratings only, check agreement with a second annotator
Bias	Old reviews, specific audience	Add recent films in the own dataset, document the limits

Table 2.2: Main risks and mitigations

2.8 Project plan and resources

The project lasts about four weeks: understanding, data and preparation (week 1), features and classical models (week 2), deep model and evaluation (week 3), deployment and report (week 4). The tools are Python, pandas, scikit-learn [6], NLTK [4], Matplotlib and Seaborn, Jupyter notebooks in VS Code, Git, and later Streamlit and Hugging Face Transformers.

Chapter 3

Data collection

3.1 Overview of the sources

Source	Raw data	Role	Tool
IMDB 50k	50,000 reviews, 25,000 per class	Training, validation, test	Public CSV, pandas
Pang & Lee	2,000 reviews, 1,000 per class	External test set	NLTK corpus
Own reviews	4,001 raw reviews with a rating	Real-world test set	TMDB API, Python requests

Table 3.1: Data sources, raw data and collection tools

3.2 Benchmark: IMDB 50k

The IMDB Large Movie Review Dataset [1] contains 50,000 reviews, 25,000 positive and 25,000 negative. Only polarized reviews are kept: a rating of 4/10 or lower is labeled negative and a rating of 7/10 or higher is labeled positive, so neutral reviews are excluded. Labels therefore come from the reviewers' own ratings and not from a third-party annotation. The collection contains at most 30 reviews per film, so that no film dominates.

The original release provides a train/test split with different films in each part. The copy used in this project is a single CSV file without a predefined split, so a stratified random split was applied (Section 5.4). As the same film can appear in several parts, scores on this split may be slightly optimistic, which the two independent sets below are meant to check.

3.3 External set: Pang & Lee

The polarity dataset v2.0 of Pang and Lee [2] contains 2,000 movie reviews, 1,000 per class. It shares no identical text with IMDB 50k. Its reviews are much longer (746 words on average, against 231 for IMDB 50k), which makes it a domain-shift test.

3.4 Own dataset

To test the system on reviews it has never seen, and to follow the requirement of collecting data, a dataset of 300 reviews was built for this project.

3.4.1 Collection

The reviews were collected through the TMDB API with a Python script based on the `requests` library. This product uses the TMDB API but is not endorsed or certified by TMDB. Two collection runs were merged. The first run listed the most popular films released since 2016. The second run listed, for each of eight genres, the most voted films and the lowest-rated films with at least 300 votes, because reviewers rate high far more often than low and negative reviews are scarce. Only reviews that carry an author rating were kept. After removing repeated review identifiers, the raw pool contains 4,001 reviews.

3.4.2 Labeling rule and filters

The label is fixed from the reviewer's own rating on a 10-point scale: ≥ 7 is positive, ≤ 4 is negative, and reviews rated between 4 and 7 are discarded. This follows the way IMDB 50k was built and keeps the labels objective. Table 3.2 shows the filters applied to the raw pool.

Step	Reviews
Raw pool (reviews with an author rating)	4,001
Duplicate texts	−6
Text not English-looking (non-ASCII share above 3%)	−9
Outside 20 to 300 words	−1,061
Film released before 2016	0
Neutral rating (between 4 and 7 out of 10)	−800
Candidates after filtering (1,491 positive, 634 negative)	2,125
Final random selection (150 positive, 150 negative)	300

Table 3.2: Filtering of the raw TMDB reviews

3.4.3 Selection

The final 300 reviews were drawn at random from the candidates, with three rules: 150 reviews per class, at most six reviews per film, and reviews spread evenly across genres. The result contains 215 distinct films released between 2016 and 2026, with an average length of 126.8 words.

Genre	Positive	Negative
Action, Animation, Comedy, Drama, Horror, Romance, Sci-Fi (each)	19	19
Thriller	17	17
Total	150	150

Table 3.3: Genre distribution of the own reviews

3.4.4 Hard cases

A review is flagged as hard when the sentiment predicted by the VADER lexicon [3] contradicts the sentiment given by the reviewer's rating. Such reviews typically contain sarcasm, negation, mixed opinions or a rating that does not match the text. The flag is automatic and was not verified manually. Because the reviews were selected at random, the share of hard cases reflects their natural frequency: 95 reviews out of 300 (32%). They are very unevenly distributed: 78 are negative reviews (52% of the negative class) and 17 are positive reviews (11% of the positive class). Negative reviews of films therefore often use vocabulary that a lexicon reads as positive.

3.4.5 Quality control

Fifty reviews (25 per class) were labeled by a second annotator from the text alone, without seeing the ratings. The annotator agreed with the rating-based label on 49 of the 50 reviews (98%), which gives Cohen's $\kappa = 0.96$. This indicates that the sentiment is clearly readable in the text and that the labels are trustworthy, within the limits of a small sample, where one disagreement changes κ by about 0.04. The single disagreement is a review rated 4/10 whose text is clearly favorable, a reminder that ratings at the boundary of the negative range can be noisy. The review was kept in the dataset, since changing the data after computing the agreement would invalidate it.

3.5 Ethics and licensing

All data are public and used for academic purposes only. Sources are cited, the TMDB reviews are not republished (the repository of the project does not contain them), and personal identifiers such as usernames are not kept.

3.6 Summary of the datasets

Dataset	Reviews	Positive / negative	Role
IMDB 50k (after deduplication)	49,582	24,884 / 24,698	Train, validation, test
Pang & Lee	2,000	1,000 / 1,000	External test
Own reviews	300	150 / 150	Real-world test

Table 3.4: Datasets used in the project

Chapter 4

Data understanding

All statistics in this chapter were computed on the full IMDB 50k file before deduplication, unless stated otherwise. The analysis was done in the notebook `01_eda.ipynb`.

4.1 Structure and quality

The file contains two columns (review text and sentiment) and no missing values. The classes are perfectly balanced (25,000 each), so accuracy is a valid metric alongside macro-F1. Deduplication on the review text finds 418 repeated texts (302 in the negative class and 116 in the positive class); none of them appears with conflicting labels. Duplicates are a risk for the evaluation: if the same review lands in both the training and the test set, the score is inflated.

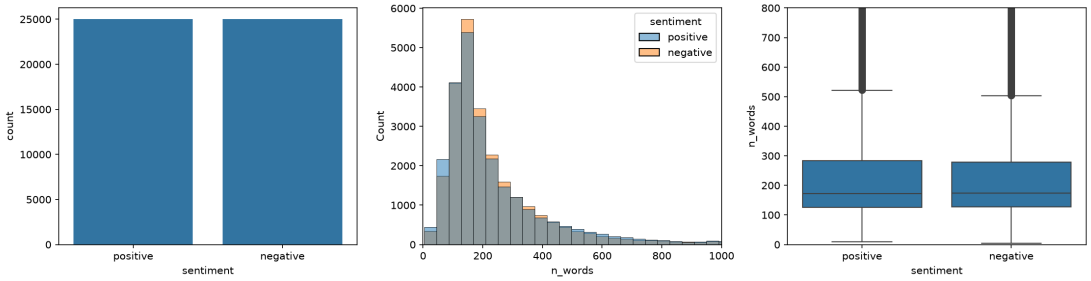


Figure 4.1: Class balance and review length distribution in IMDB 50k

4.2 Review length

Class	Mean	Std	Min	25%	Median	75%	Max
Negative	229.5	164.9	4	128	174	278	1,522
Positive	232.8	177.5	10	125	172	284	2,470

Table 4.1: Review length in words by class (IMDB 50k)

Both classes have almost the same length distribution, so length alone does not reveal the sentiment. The distribution has a long right tail: the 95th percentile is 590 words and the 99th is 908 words. This matters later for models that limit the length of their input.

4.3 Noise

HTML line breaks (`<br />`) appear in 58.4% of the reviews and non-ASCII characters in 9.3%. Both must be handled during preparation.

4.4 Vocabulary

Words such as “movie”, “film”, “like” and “just” are among the most frequent in both classes and carry no sentiment. To find the words that actually separate the classes, a logistic regression was fitted on TF-IDF features and its largest coefficients were inspected (Figure 4.2). This model is only a lens to explore the vocabulary; it is trained on all the data and is not an evaluation. The most negative words are “worst”, “waste”, “awful”, “bad”, “boring”, “poor”, “terrible” and “dull”. The most positive are “great”, “excellent”, “best”, “perfect”, “wonderful”, “amazing”, “loved” and “brilliant”.

Bigrams add information. “waste time” appears 1,453 times among the most frequent negative bigrams, while “10 10” (a rating written in the text) appears 734 times among the positive ones. Other frequent bigrams, such as “special effects” and “sci fi”, appear in both classes and carry no sentiment on their own.

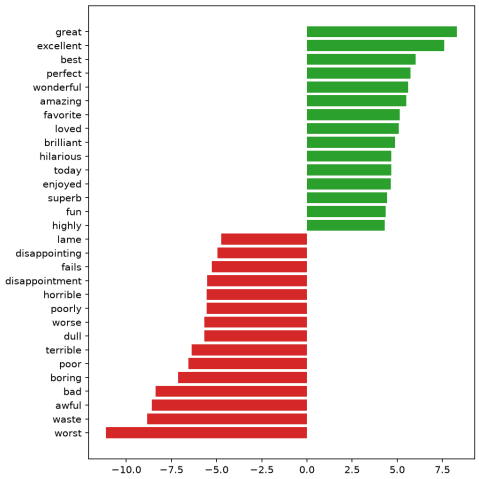


Figure 4.2: Words with the largest logistic regression coefficients (red: negative, green: positive)

4.5 Negation

At least one negation token (“not”, “no”, “never” or a contraction ending in “n’t”) appears in 87.4% of the reviews. Handling negation correctly is therefore a central preparation issue: removing it would turn “not good” into “good”.

4.6 Comparison of the three datasets

Dataset	Reviews	Positive share	Mean length (words)
IMDB 50k (deduplicated)	49,582	50.2%	231.4
Pang & Lee	2,000	50.0%	746.3
Own reviews	300	50.0%	126.8

Table 4.2: Size, balance and length of the three datasets

The three datasets differ strongly in length: Pang & Lee reviews are more than three times longer than IMDB reviews, and the own reviews, shortened by the 20 to 300 word filter, are the shortest. They also come from different periods and sources. A lower performance on both independent sets is therefore expected (Figure 4.3).

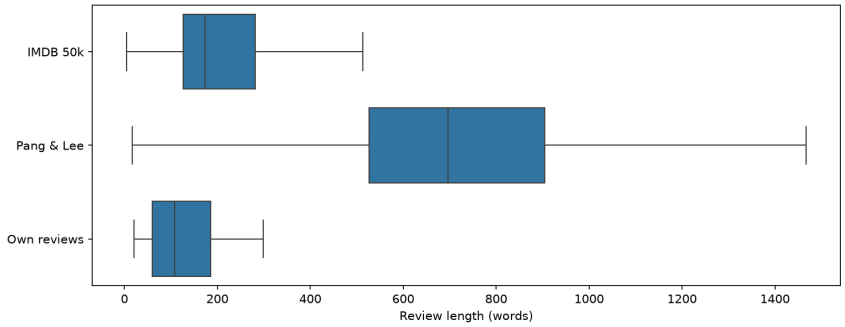


Figure 4.3: Review length in the three datasets (outliers hidden)

4.7 Hard cases in the own reviews

The VADER flag marks 95 of the 300 own reviews as hard, 78 of them negative and 17 positive.

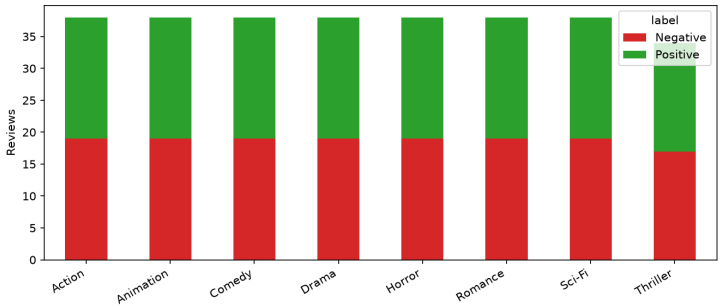


Figure 4.4: Own reviews by genre and class

4.8 Summary of findings

Classes are balanced and labels reliable. Noise must be cleaned (HTML tags, punctuation, 418 duplicates), negation is present in most reviews and must be preserved, and bigrams carry useful information. The three datasets differ strongly in length, so a performance drop on the independent sets is expected.

Chapter 5

Data preparation

The preparation keeps the sentiment signal (in particular negations), avoids any information leaking from the test set, and is reproducible through a single function and a fixed random seed.

5.1 Pipeline overview

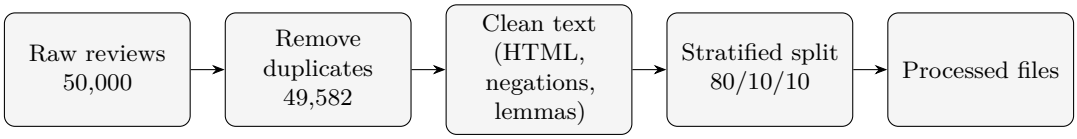


Figure 5.1: Data preparation pipeline for the IMDB benchmark

5.2 Deduplication

418 duplicated reviews (302 negative, 116 positive) were removed, leaving 49,582 reviews: 24,884 positive and 24,698 negative. This is done before splitting, otherwise the same text could appear in both the training and the test set.

5.3 Text cleaning

The `clean()` function (Appendix A) applies the steps of Table 5.1. Lemmatization uses the WordNet lemmatizer [5] through NLTK. Without part-of-speech information a few forms are reduced oddly ("was" becomes "wa"), which does not affect the models.

Step	Operation	Reason
HTML removal	Delete tags such as ` `	Present in 58.4% of reviews, no sentiment
Lowercasing	Convert all text to lowercase	Merge “Great” and “great”
Contractions	“won’t” → “will not”, “can’t” → “can not”, “n’t” → “ not”	Turn negations into a separate token
Non-letters	Replace digits and punctuation by spaces	Reduce noise and vocabulary size
Lemmatization	Reduce words to their dictionary form	Merge “movies” and “movie”
Short tokens	Remove one-letter tokens	Residues such as “s” or “t”

Table 5.1: Cleaning steps

Stop words are not removed in the final pipeline. The default NLTK list contains negations (“not”, “no”, “isn”, “don” and others), and removing them changes the meaning of a sentence, as the examples show.

Input	Final pipeline	With default stop-word removal
This isn’t a good movie, I can’t recommend it.	this is not good movie can not recommend it	good movie recommend
Not bad at all	not bad at all	bad
Loved it!!! 10/10, wouldn’t change a thing.	loved it would not change thing	loved would change thing

Table 5.2: Examples of cleaning, with and without default stop-word removal

5.4 Training, validation and test split

After deduplication, the data were split with stratification on the label and a fixed seed (42) into 80% training, 10% validation and 10% test. The test set is used only once, for the final evaluation.

Set	Reviews	Positive	Negative
Training	39,665	19,907	19,758
Validation	4,958	2,488	2,470
Test	4,959	2,489	2,470

Table 5.3: Class distribution of the three sets

5.5 Integration of the three sources

The Pang & Lee reviews and the own reviews are processed with exactly the same `clean()` function as IMDB, so that every source is cleaned in the same way, and they are kept apart as external test sets. They are never used for training or tuning. Checks confirm that the training, validation and test sets share no review, and that neither external set contains a review of IMDB 50k.

5.6 Enrichment

The own reviews are enriched with the film title, genre and release year provided by TMDB, and with the VADER-based hard-case flag (Section 3.4). These columns are not used for training; they allow a detailed error analysis by genre and by difficulty in the evaluation phase.

5.7 Comparison of cleaning variants

Five variants were compared using a TF-IDF representation (minimum document frequency 3, sublinear term frequency) and a logistic regression classifier ($C = 4$), trained on the training set and evaluated on the validation set only. This baseline serves only to compare cleaning choices; feature engineering and modeling are the object of the next phase.

Variant	F1 (unigrams)	F1 (unigrams + bigrams)
A: raw text, lowercase only	0.8985	0.9141
B: HTML and punctuation cleaned	0.9003	0.9104
C: B + lemmatization	0.8981	**0.9155**
D: C + default stop words removed (negations lost)	0.8969	0.9050
E: C + stop words removed, negations kept	0.8991	0.9110

Table 5.4: Validation macro-F1 of the five cleaning variants

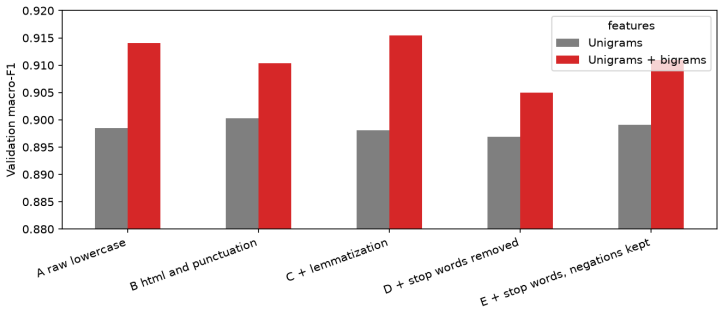


Figure 5.2: Validation macro-F1 by cleaning variant (the vertical axis starts at 0.88)

Interpretation. With 4,958 validation reviews, the standard error of an F1 estimate is about 0.4 points. Variants A, B, C and E are therefore statistically close. Removing stop words brings no gain, and removing the negations as well (variant D) is the only clear loss: 0.6 to 1.1 points with bigrams. Cleaning matters less for a linear model than for the choice of features, but it must never destroy negation.

Decision. Variant C was kept: HTML removal, contraction expansion, punctuation removal and lemmatization, with all stop words preserved. A second column without stop words (variant E) was saved for later comparison.

Result. The cleaning reduces the number of distinct tokens in the training set from about 340,000 to about 82,000, and the average review length from 231.5 words to 219.2 tokens. The stage produces five files (`train.csv`, `val.csv`, `test.csv`, the processed external set and the processed own reviews), each with the columns `review`, `clean`, `clean_nostop` and `label`. They are the input of the feature engineering phase.

Chapter 6

Conclusion and next steps

6.1 Summary of the phase

- The business problem, objectives, success criteria and risks are defined (macro-F1 ≥ 0.88, under one second per review, working demonstration).
- Three sources are collected: IMDB 50k, Pang & Lee and 300 own reviews built from 4,001 raw TMDB reviews, with labels confirmed on a sample ($\kappa = 0.96$).
- The exploration identified the data quality issues (418 duplicates, HTML in 58.4% of reviews, 87.4% of reviews with a negation) and the differences between sources (231, 746 and 127 words on average).
- The preparation produced clean, leakage-free training, validation and test sets, with a cleaning pipeline chosen by experiment and negations preserved.

6.2 Limitations

The IMDB split is random, so the same film can appear in several parts and validation scores may be slightly optimistic. The own dataset is small (300 reviews) and its agreement check uses 50 reviews. The hard-case flag is automatic and not verified manually. Only English reviews of films are covered. Nothing has been evaluated on the test sets yet.

6.3 Next steps

1. **Feature engineering:** compare Bag-of-Words, TF-IDF and bigrams with a fixed classifier on the validation set.
2. **Modeling:** Naive Bayes, Logistic Regression and Linear SVM with cross-validated tuning, then one deep model (BiLSTM or DistilBERT).
3. **Evaluation:** one evaluation on the IMDB test set, on Pang & Lee and on the own reviews, with an error analysis by category and by hard case.
4. **Deployment:** a Streamlit application that reuses the same `clean()` function.

Bibliography

[1] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng and C. Potts, Learning Word Vectors for Sentiment Analysis, *Proc. ACL*, 2011, pp. 142–150.

[2] B. Pang and L. Lee, A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts, *Proc. ACL*, 2004.

[3] C. J. Hutto and E. Gilbert, VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text, *Proc. ICWSM*, 2014.

[4] S. Bird, E. Klein and E. Loper, *Natural Language Processing with Python*, O'Reilly, 2009.

[5] G. A. Miller, WordNet: A Lexical Database for English, *Communications of the ACM* 38(11), 1995, pp. 39–41.

[6] F. Pedregosa et al., Scikit-learn: Machine Learning in Python, *JMLR* 12, 2011, pp. 2825–2830.

Appendix A

Code and reproduction

A.1 Preparation code

```
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

NEG = {"no", "not", "nor", "never", "don", "didn", "doesn", "isn", "wasn",
       "aren", "weren", "couldn", "wouldn", "shouldn", "won", "hasn",
       "haven", "hadn", "ain", "mustn", "needn", "shan", "mightn"}
STOP_ALL = set(stopwords.words("english"))
STOP_KEEP = STOP_ALL - NEG
LEM = WordNetLemmatizer()

def clean(text, stop="none", lemmatize=True):
    text = re.sub(r"<.*?>", "␣", text).lower()
    text = text.replace("won't", "will␣not").replace("can't", "can␣not")
    text = re.sub(r"n't\b", "␣not", text)
    text = re.sub(r"[^a-z\s]", "␣", text)
    tokens = text.split()
    if stop == "all":
        tokens = [t for t in tokens if t not in STOP_ALL]
    elif stop == "keep_neg":
        tokens = [t for t in tokens if t not in STOP_KEEP]
    if lemmatize:
        tokens = [LEM.lemmatize(t) for t in tokens]
    return "␣".join(t for t in tokens if len(t) > 1)
```

```
imdb = imdb.drop_duplicates(subset="review").reset_index(drop=True)
train, rest = train_test_split(imdb, test_size=0.2, stratify=imdb["label"],
    random_state=42)
val, test = train_test_split(rest, test_size=0.5, stratify=rest["label"],
    random_state=42)
```

A.2 Structure of the deliverable

```
Sentiment_Analysis_Phase1/
```

```
report/       main.tex, report.pdf
notebooks/    00_data_collection.ipynb, 01_eda.ipynb,
              02_data_preparation.ipynb
src/          __init__.py, preprocess.py
data/raw/     imdb_50k.csv, pang_lee_2000.csv
data/own/     tmdb_raw.csv, own_reviews.csv,
              to_annotate.csv, annotated.csv
figures/      01_balance_length.png ... 05_cleaning_variants.png
README.md, DATA.md, requirements.txt
```

A.3 How to reproduce

1. Install the dependencies: `pip install -r requirements.txt`.
2. Set the environment variable `TMDB_KEY` with a personal TMDB API key (only needed to collect the TMDB reviews again; the raw file is provided).
3. Run the notebooks in order: `00_data_collection`, `01_eda`, `02_data_preparation`. The project folder must be the root of the notebooks.

The source code is also available at `https://github.com/yacine-saidane/Sentiment-Analysis` (without the data files).